

Custom Hooks

hooks are functions that let you “hook into” React state and lifecycle features from **functional components**. React provides built-in hooks like:

A **custom hook** is **your own reusable function** that uses React hooks internally to encapsulate and share logic between components.

Rule: A custom hook always starts with the prefix `use`.

This is how React can identify it as a hook and enforce the **Rules of Hooks**.

Why Use Custom Hooks?

Custom hooks exist to:

1. **Reuse logic** — Instead of duplicating the same `useState` + `useEffect` logic in multiple components, you can extract it.
2. **Abstract complexity** — Hide implementation details and provide a clean API to your components.
3. **Maintain separation of concerns** — Components stay focused on rendering; hooks handle logic.

```
import { useState, useEffect } from "react";
```

```
// Custom hook
```

```
function useWindowWidth() {  
  const [width, setWidth] = useState(window.innerWidth);
```

```
  useEffect(() => {  
    const handleResize = () => setWidth(window.innerWidth);  
    window.addEventListener("resize", handleResize);
```

```
// Cleanup
```

```
  return () => window.removeEventListener("resize", handleResize);  
}, []);
```

```
  return width;
```

```
}
```

```
export default useWindowWidth;
```

Using the hook in a component:

```
import React from "react";
```

```
import useWindowWidth from "../useWindowWidth";
```

```
function MyComponent() {
```

```
  const width = useWindowWidth();
```

```
  return <div>Window width: {width}px</div>;
```

```
}
```

```
export default MyComponent;
```

A custom hook can internally use **any built-in hook**:

Q. Why would you use a custom hook instead of a component?

Answer:

Components handle **UI**, while custom hooks handle **logic**. Custom hooks allow you to **share logic** like fetching data, form handling, or event listeners **without duplicating code**.

Using a custom hook keeps components **focused on rendering** and makes logic **reusable** across multiple components.

Q. Can you call a custom hook inside a loop, condition, or nested function?

Answer:

No. Custom hooks follow the **Rules of Hooks**:

1. Only call hooks at the **top level** of a component or another hook.
2. Never call inside loops, conditions, or nested functions.

Rules for Hooks

1. Only call hooks at the top level

-  Call hooks inside React functional components or custom hooks.
-  Do not call hooks inside loops, conditions, or nested functions.

Why: React relies on call order to track state and effects. Violating this breaks state consistency.

2. Only call hooks from React functions

-  Functional components or other custom hooks.
-  Do not call hooks from regular JS functions, class components, or event handlers outside a hook.

Why: Hooks need the React rendering context to work.

3. Custom hooks must start with use

- Example: useFetch, useForm, useToggle
- React detects hooks by the use prefix and enforces the rules.

Why: This prevents accidental misuse and allows linting rules (eslint-plugin-react-hooks) to check correct usage.

4. Effects cleanup rule

- Always clean up subscriptions or timers inside useEffect return function.
- Example: remove event listeners, cancel fetch requests, clear intervals.

```
useEffect(() => {  
  const timer = setInterval(doSomething, 1000);  
  return () => clearInterval(timer);  
}, []);
```

Why: Prevents memory leaks and updating state on unmounted components.

5. Do not conditionally call hooks

-  Wrong:

```
if (isOpen) useEffect(() => { ... }, []);
```

-  Correct:

```
useEffect(() => {  
  if (isOpen) { ... }  
}, [isOpen]);
```

Why: Conditional calls change the **hook call order**, breaking React's internal hook tracking.